

리눅스 시스템 프로그래밍

7장 프로세스간 통신



목차

1. 프로세스간 통신
2. 파이프
3. 메시지 큐
4. 공유메모리

프로세스간 통신

1. 통신 메소드
2. System V IPC와 POSIX IPC
3. IPC 관련 도구

통신 메소드

- 리눅스에서 프로세스란 독립적으로 실행되는 객체로 프로세스간의 통신(Inter Process Communication, 이하 IPC)을 위해서는 별도의 자원을 할당 받아야 한다. 리눅스 커널에서는 다음과 같은 IPC 메소드를 제공한다.
 - 파이프
 - 메세지 큐
 - 세마포
 - 공유 메모리
- IPC 메소드 특징

IPC 메소드	통신 방향	관련 프로세스	특징
파이프	단방향	2개의 프로세스간 통신	PIPE (4096 bytes)크기 고려
메시지 큐	양방향	2개 이상의 프로세스간 통신 가능	짧은 메시지(1024 bytes 미만) 교환에 적절
공유메모리	양방향	2개 이상의 프로세스간 통신 가능	긴 메시지 교환에 적절, 통신시 충돌문제 해결 필요
세마포	양방향	2개 이상의 프로세스간 통신 가능	주로 IPC 동기화 요소로 사용

SYSTEM V IPC 와 POSIX IPC

- IPC 제어를 위한 시스템 콜(API)에는 두 가지 표준이 있다. 하나는 오래된 버전의 System V IPC이고, 다른 하나는 나중에 표준화된 POSIX IPC다. System V IPC는 오랜 역사를 가진 만큼 이 기종간 코드 호환성을 확실히 보장해 주지만 API의 형식이 직관적이지 않아 처음 사용 시 다소 불편한 반면 POSIX IPC는 직관적으로 API가 구성되어 있어서 좀 더 사용하기 쉽다.

■ System V IPC 객체

- System V IPC 객체란 메시지 큐, 공유 메모리 세마포 등을 말하는 것으로 한 시스템 내에서만 이용가능하며 이 객체는 프로세스를 종료하더라도 강제로 삭제하지 않는 한 계속 살아있고 각 객체는 IPC ID 라는 식별자에 의해 구분이 되며, 이 식별자의 기준은 IPC key에 의해 결정된다
- 이 세가지 객체는 **xxxget**, **xxxctl** 형식의 시스템 콜에 의해 생성 및 제어된다.

예를 들면 다음과 같다.

msgget – 메시지큐 객체 생성자 함수

shmget – 공유메모리 객체 생성자 함수

semget – 세마포 객체 생성자 함수

IPC 관련 도구

■ ipcs 명령

현재 리눅스 시스템에 존재하는 IPC 객체의 상태를 확인할 수 있는 명령으로 다음은 이 명령의 수행 예이다.

```
user@ubuntu: ~  
user@ubuntu:~$ ipcs  
  
----- Shared Memory Segments -----  
key          shmid        owner         perms         bytes         nattch        status  
0x00000000   1441792      user          600           524288        2             dest  
0x00000000   393217       user          600           524288        2             dest  
0x00000000   425986       user          600           524288        2             dest  
0x00000000   1474563      user          600           1048576       2             dest  
0x00000000   720900       user          600           524288        2             dest  
0x00000000   753669       user          600           16777216     2             dest  
0x00000000   1179654      user          600           33554432     2             dest  
0x00000000   950279       user          600           524288        2             dest  
0x00000000   1048584      user          600           524288        2             dest  
0x00000000   1114121      user          600           2097152      2             dest  
0x00000000   1277962      user          600           524288        2             dest  
0x00000000   1572875      user          600           524288        2             dest  
0x00007770   7733260     user          666           16384         0             dest  
0x00000000   7831565      user          600           393216        2             dest  
0x00000000   9404430      user          600           393216        2             dest  
0x00000000   9437199      user          600           524288        2             dest  
0x00000000   11960336     user          600           524288        2             dest  
  
----- Semaphore Arrays -----  
key          semid         owner         perms         nsems  
  
----- Message Queues -----  
key          msqid         owner         perms         used-bytes   messages  
  
user@ubuntu:~$
```

IPC 관련 도구

- ipcrm

IPC 통신 프로그램을 시험하다가 오류가 발생하여 프로그램이 비정상적으로 종료되면 IPC 오브젝트를 정상적으로 반납할 수 없게 된다. 이럴 때에는 다음과 같은 명령을 이용하여 삭제할 수 있다.

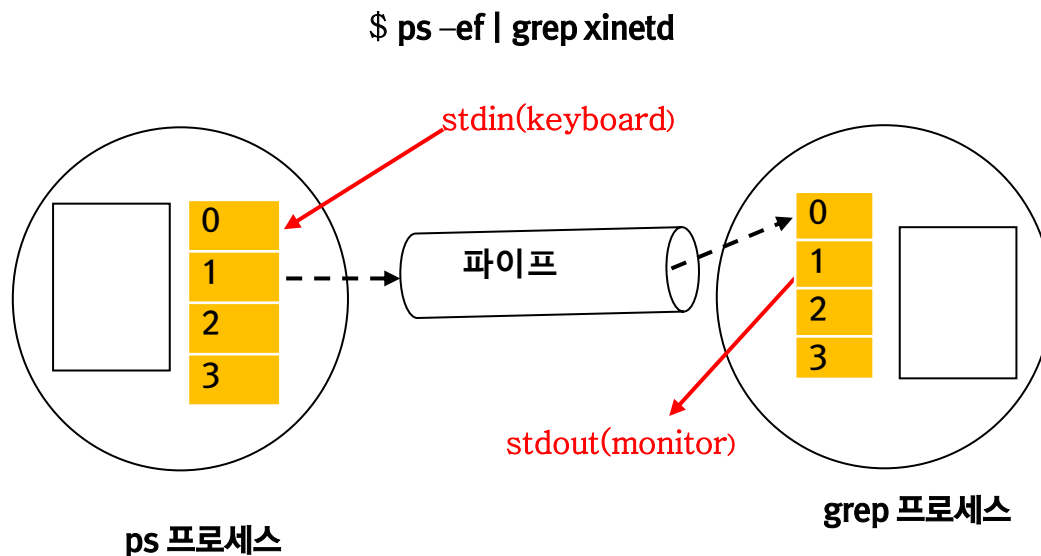
```
$ ipcrm -q <messagequeue_id>  
      [-m <sharedmemory_id> 또는 -s <semaphore_id>]
```

파이프

- 1.파이프란?
2. Anonymous pipe
3. Named pipe

파이프란

- 파이프란 두 프로세스간에 데이터를 전달하기위해 사용하는 메소드로 셸에서 명령어를 결합할 때 사용하는 ‘|’ 가 대표적인 예이다. 이 파이프는 한 프로세스의 출력 스트림을 다른 프로세스의 입력 스트림으로 연결해주는 방법으로 다음 그림을 통해 파이프의 동작 원리를 쉽게 이해할 수 있다.



셸의 파이프 동작 원리

- 같은 파이프 기능은 어떻게 구현할 수 있을까? 파이프는 크게 익명의 anonymous pipe와 named pipe로 구분할 수 있다. 이제 이 구현 방법을 살펴보자.

ANONYMOUS PIPE

- `popen()`, `pclose()` 함수 이용

```
#include <stdio.h>
```

```
FILE *popen(const char *command, const char  
*open_mode);  
int pclose(FILE *stream_to_close);
```

- `popen()`과 `pclose()` 함수는 파이프를 구현하기 위한 가장 간단한 함수로, 첫번째 인수 `command`에 현재 이 프로세스와 통신할 프로그램의 이름을 지정한다. 두 번째 인수에는 `fopen()` 함수에서 사용되는 모드 중 “r” 이나 “w” 를 설정한다.
- 이 함수는 성공하면 파이프에 대한 파일포인터를 반환한다. 이 파일 포인터를 사용하면 파일 입출력 함수인 `fread()`, `fwrite()` 함수를 이용하여 메시지를 파이프를 통해 전달할 수 있다. `pclose()` 함수는 파일포인터의 연결을 종료시키는 함수이다.

ex01.c 소스를 확인하고 실행해 봅시다.

소스코드 (ex01.c)

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    FILE *fp;
    int m;

    if((fp=popen("grep 'Hello'", "w")) == NULL) {
        fprintf(stderr, "popen failed.\n");
        exit(1);
    }
    for( m = 1; m <= 10; m++ ) {
        fprintf(fp, "Hello, pipe!!\n");
        fprintf(fp, "Goodbye, pipe...\n");
    }
    pclose(fp);
    return(0);
}
```

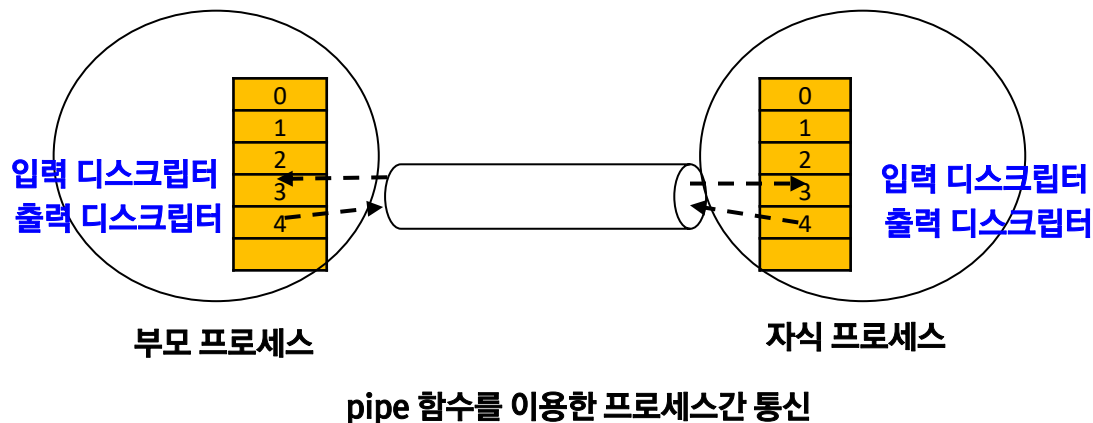
ANONYMOUS PIPE

■ pipe() 함수의 이용

```
#include <unistd.h>
```

```
int pipe(int file_descriptor[2]);
```

- pipe() 함수를 호출하면 파이프를 생성하고 file_descriptor 배열에 파이프 디스크립터(파일 디스크립터)를 두 개 받아온다. 이 중 file_descriptor[0]에는 파이프의 입력 디스크립터, file_descriptor[1]에는 파이프의 출력 디스크립터를 전달 받는다. 즉 file_descriptor[1]에 메시지를 출력하면 file_descriptor[0]을 통해 입력 받을 수 있다. 이 정보를 공유할 수 있는 프로세스, 즉 자신이나 자식 프로세스와는 이 파이프 디스크립터를 이용하여 메시지를 전달할 수 있다.



ex02.c 소스를 확인하고 실행해 봅시다.

소스코드 (ex02.c)

```
#define MAXSIZE 4096
```

```
int main(void) {
```

```
    int pd[2];
```

```
    pid_t pid;
```

```
    int n;
```

```
    char buf[MAXSIZE];
```

```
    if( pipe(pd) == -1 ) {
```

```
        perror("pipe");
```

```
        exit(1);
```

```
    }
```

```
    switch(pid=fork()) {
```

```
    case -1:
```

```
        perror("fork");
```

```
        exit(1);
```

```
        break;
```

```
    case 0:
```

```
        close(pd[0]);
```

```
        strcpy(buf, "Hello, Parent!!");
```

```
        write(pd[1], buf, strlen(buf));
```

```
        exit(1);
```

```
        break;
```

```
    default:
```

```
        close(pd[1]);
```

```
        n=read(pd[0], buf, MAXSIZE);
```

```
        buf[n]='\0';
```

```
        close(pd[0]);
```

```
        printf("PARENT : %s from CHILD\n", buf);
```

```
        if(waitpid(pid, NULL, 0)==-1) {
```

```
            perror("waitpid");
```

```
            exit(1);
```

```
        }
```

```
        break;
```

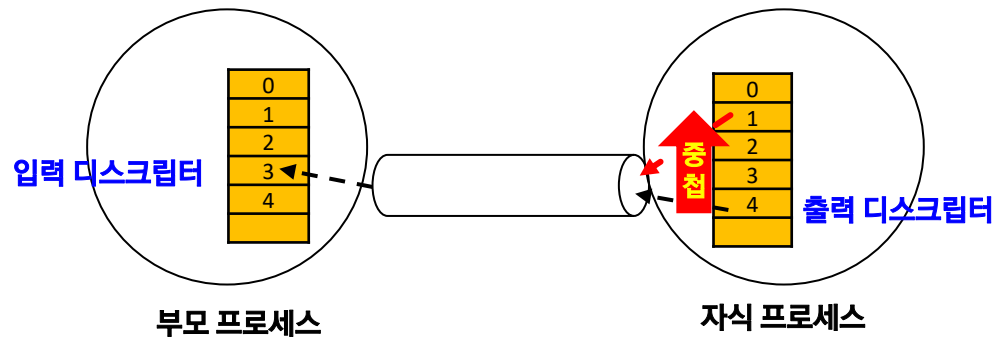
```
    }
```

```
    return 0;
```

```
}
```

ANONYMOUS PIPE 와 중첩

- 앞에서 살펴본 ex02.c 에 다음과 같이 파일 중첩 함수 dup()를 이용하여 표준 입출력 디스크립터와 파이프를 중첩시켜 보시다.



파이프와 표준 입출력 디스크립터의 중첩



ex02.c 를 ex03.c 로 복사하고 dup() 또는 dup2() 함수를 이용하여 자식 프로세스에서 실행한 결과를 부모 프로세스에서 출력하도록 수정해 봅시다. 실행이 잘 되었다면 자식 프로세스에서 “ls” 명령을 실행하도록 수정하고 다시 실행해봅시다.

NAMED PIPE

- named pipe란 말 그대로 이름이 있는 파이프를 말한다. 앞서 1.2에서 살펴보았던 파이프는 프로세스의 수행이 끝나면 함께 소멸되었다. 그러나 프로세스가 종료되더라도 여전히 존재하며 언제라도 이 이름으로 파이프 통신을 할 수 있는 자원을 말한다. 리눅스에서는 이를 파이프 파일, 혹은 FIFO 파일이라고 한다.

- 파이프 파일을 생성할 수 있는 방법은 다음과 같다.

```
$ mkfifo <filename>
```

혹은

```
$ mknod <filename> p
```

- 또는 시스템 콜 `mkfifo()` 함수를 이용할 수도 있다. 자세한 사용법은 온라인 매뉴얼을 참조하도록 한다.
- 이렇게 작성된 파이프 파일은 일반 파일을 처리하는 방법과 동일한 방법으로 접근할 수 있다. 즉, `open()`, `read()`, `write()`, `close()` 함수 등을 이용하여 파이프 통신이 가능하다.

ex04-1.c 과 ex04-2.c 소스를 확인하고 실행해 보시다 결과를 확인한 후 myfifo 파일을 삭제한 후 mkfifo() 함수로 생성하여 다시 실행해 보시다.

소스코드 (ex04-1.c)

```
int main(void)
{
    int pd, n;
    char msg[1024];

    if(mkfifo("./myfifo", 0600)==-1){
        if(errno!=EEXIST){
            perror("mkfifo");
            exit(1);
        }
    }
    if((pd=open("./myfifo", O_WRONLY)) == -1){
        perror("open");
        exit(1);
    }
    while(1){
        printf("namedpipe1> ");
        fgets(msg, 1023, stdin);
        if(strncmp(msg, "quit", 4)==0)
            break;
        if((n=write(pd, msg, strlen(msg)))==-1){
            perror("write");
            exit(1);
        }
    }
    close(pd);
    return(0);
}
```

소스코드 (ex04-2.c)

```
int main(void)
{
    int pd, n = 0;
    char rcvmsg[1024];

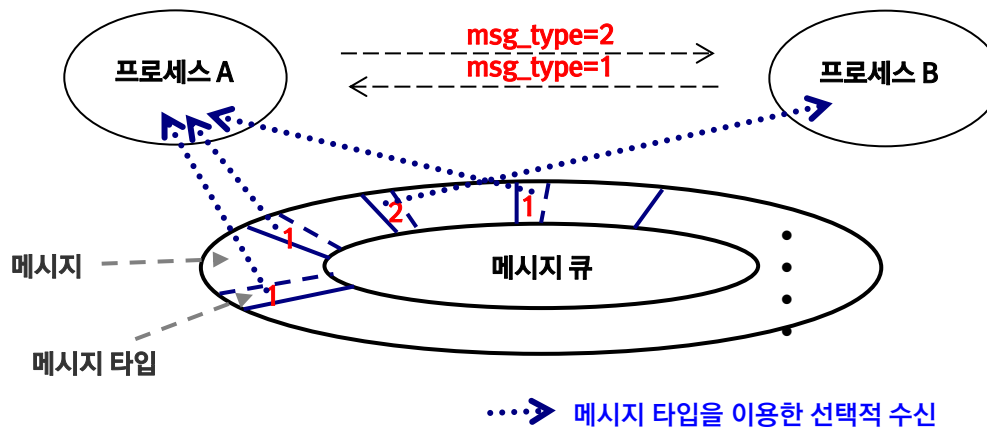
    if(mkfifo("./myfifo", 0600)==-1){
        if(errno!=EEXIST){
            perror("mkfifo");
            exit(1);
        }
    }
    if((pd=open("./myfifo", O_RDONLY)) == -1 ){
        perror("open");
        exit(1);
    }
    while((n=read(pd, rcvmsg, 1023)))>0){
        rcvmsg[n]='\0';
        write(1, "namedpipe2> ", 12);
        write(1, rcvmsg, n );
    }
    if(n==-1){
        perror("read");
        exit(1);
    }
    write(1, "\n", 1 );
    close(pd);
    return(0);
}
```

메시지 큐

1. 메시지 큐란?
2. 메시지 큐 함수

메시지 큐 란?

- IPC 기능 중 메시지 큐는 앞서 살펴본 파이프와 유사하지만 파이프와 같이 단 방향(one-way) 통신만 가능한 것이 아니라 양 방향(both-way) 통신이 가능하며 두 개 이상의 프로세스간에 통신을 할 수 있는 방법이다.
- 하나의 메시지 큐를 여러 프로세스가 이용해야 하므로 구분자가 필요한데 이를 메시지 타입이라고 한다. 다음 그림에서 처럼 프로세스에서는 메시지 큐로부터 메시지 타입이 일치하는 것만 선택적으로 수신할 수 있어 혼돈 없이 메시지를 교환할 수 있다.



메시지 큐 사용 예

메시지 큐 함수

■ 메시지큐 생성자 함수

- msgget() 함수는 메시지 큐를 생성하는 함수로 다음과 같은 원형을 갖고 있다.

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
```

```
int msgget(key_t key, int msgflag);
```

첫 번째 인수 **key**에는 키 값을 전달해야 한다. 이 키는 메시지 큐를 이용하여 통신하려는 프로세스끼리는 동일한 값을 지정해야 한다. 키 값은 **ftok()** 함수 또는 직접 값을 설정한다. 키 값에 **IPC_PRIVATE**를 지정하면 현재의 프로세스만 접근할 수 있는 메시지 큐를 생성한다.

두 번째 인수 **msgflag**에는 **IPC_CREAT**와 메시지 큐의 사용 권한을 비트 OR하여 지정한다. 이외의 플래그는 거의 사용되지 않는다. 만약 동일한 키 값의 메시지 큐가 이미 존재하면 **IPC_CREAT** 플래그는 무시된다.

이 함수의 호출이 성공하면 메시지 큐의 오브젝트 ID를 반환하고, 실패하면 -1을 반환한다.

메시지 큐 함수

■ 메시지 큐 송수신 함수

- `msgsnd()` 함수는 메시지 큐로 메시지를 전송할 때 사용하는 함수로 다음과 같은 원형으로 이루어져 있다.

```
int msgsnd(int msgid, const void *msg_ptr, size_t msg_sz, int msg_flg);
```

첫 번째 인수 `msgid`에는 `msgget()` 함수에서 반환 받은 메시지 큐의 오브젝트 ID를,

두 번째 인수 `msg_ptr`에는 메시지 큐에 전송하려는 메시지의 주소를,

세 번째 인수 `msg_sz`에는 전송하려는 메시지의 길이를 지정한다.

네 번째 인수 `msg_flg`에는 0 이나 `IPC_NOWAIT` 플래그를 설정할 수 있다. `msg_flg` 플래그가 0 으로 설정되어 있을 때 메시지 전송 시 메시지 큐가 완전히 차 있으면 `msgsnd()` 함수에서 블록 되어 프로세스의 수행이 멈추게 된다. 만약 `IPC_NOWAIT`로 설정하면 메시지 큐가 꽉 차있을 때라도 프로세스의 수행이 멈추지 않고 다른 처리를 진행할 수 있다. 이 때 `msgsnd()` 함수의 반환 값은 -1이고 `errno` 변수에 `EAGAIN` 이 설정된다.

메시지 큐 함수

- 여기에서 메시지 큐에 메시지를 보낼 때에는 반드시 메시지 앞에 메시지 타입을 붙여 보내야 한다. 그 이유는 msgrcv() 함수에서 이 타입을 이용하기 때문이다. 따라서 일반적으로 다음과 같은 구조체 자료형을 이용하여 전송할 메시지를 작성한다.

```
struct msgtype {  
    long msg_type;  
    char msg_text[1024];  
};
```

- 이 때 주의해야 할 사항은 msg_type 멤버가 반드시 구조체 앞쪽에 선언되어야 한다.

메시지 큐 함수

- `msgrcv()` 함수는 메시지 큐로부터 메시지를 수신 받는 함수로 그 원형은 다음과 같다.

```
int msgrcv(int msgid, void *msg_ptr, size_t msg_sz, long msgtype, int msgflg);
```

- 여기에서 네 번째 인수 `msgtype`을 제외하고 나머지 인수들은 `msgsnd()` 함수의 인수들과 동일하다. `msgtype`에는 메시지 큐로부터 수신할 메시지 타입을 지정한다. 만약 이 부분에 0을 지정하면 메시지 큐에 등록된 순서대로 메시지를 수신하고, 0보다 큰 양수를 지정하면 그 타입에 해당하는 메시지만 선택적으로 수신한다. 서로 통신할 프로세스끼리 메시지 타입을 정하고 송수신시 이 타입을 이용하면 하나의 메시지 큐를 이용하여 여러 프로세스 간에 통신을 할 수 있다.
- `msgsnd()` 함수와 마찬가지로 다섯번째 인수인 `msgflg`에 0 또는 `IPC_NOWAIT`를 설정할 수 있는데, 0은 메시지 큐가 비어있는 경우 프로세스는 수행을 멈추고 다른 프로세스에서 메시지를 전송할 때까지 기다린다. 만약 `IPC_NOWAIT`를 설정하면 큐가 비어있을 지라도 프로세스가 수행을 멈추지않고 계속 진행시킨다. 이 때 `msgrcv()` 함수의 반환 값은 -1, `errno` 변수에는 `ENOMSG`가 설정된다.

메시지 큐 함수

■ 메시지 큐 제어 함수

- 메시지 큐를 제어할 수 있는 함수는 msgctl 함수로 그 원형은 다음과 같다

```
int msgctl(int msgid, int command, struct msqid_ds *buf);
```

```
struct msqid_ds {  
    uid_t msg_perm.uid;  
    uid_t msg_perm.gid;  
    mode_t msg_perm.mode;  
};
```

첫 번째 인수 msgid에는 msgget() 함수의 반환 값인 메시지 큐 오브젝트의 ID를 지정한다.

두 번째 인수 command에는 제어 명령을 지정한다. 제어 명령의 종류는 다음과 같다.

제어 명령	의미
IPC_STAT	메시지 큐에 관련된 정보를 세번째 인수로 읽어온다. struct msqid_ds 구조체 이용
IPC_SET	메시지 큐에 관련된 정보를 세번째 인수에 전달한 값으로 설정한다. struct msqid_ds 구조체 이용
IPC_RMID	메시지 큐 오브젝트를 삭제한다.

세 번째 인수 buf에는 제어 명령에 따라 받아올 값을 저장할 구조체 포인터를 전달한다.

ex05-1.c 과 ex05-2.c 소스를 확인하고 실행해 봅시다

소스코드 (ex05-1.c)

```
int main(void)
{
    int msgid;
    char buf[1024];
    struct mymsgbuf {
        long m_type;
        char m_str[1024];
    } sendbuf;

    if((msgid=msgget(0x123400,IPC_CREAT|0666))== -1)
    {
        perror("msgget");
        exit(1);
    }
    while (1)
    {
        printf(" Input --> ");
        fgets(buf, 1023, stdin);
        sendbuf.m_type=1;
        strcpy(sendbuf.m_str,buf);
        msgsnd(msgid,&sendbuf, strlen(sendbuf.m_str),0);
        if(!strncmp(buf,"end",3))
            break;
    }
    sleep(1);
    return 0;
}
```

소스코드 (ex05-2.c)

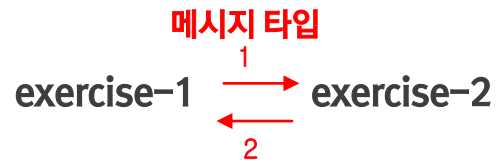
```
int main(void)
{
    int msgid,n;
    struct mymsgbuf {
        long m_type;
        char m_str[1024];
    } recvbuf;

    if((msgid=msgget(0x123400,IPC_CREAT|0666))== -1)
    {
        perror("msgget");
        exit(1);
    }
    while (1)
    {
        n=msgrcv(msgid,&recvbuf,1024, 1,0);
        recvbuf.m_str[n]='\0';
        printf(">> %s\n",recvbuf.m_str);
        if(!strncmp(recvbuf.m_str,"end", 3))
            break;
    }
    msgctl(msgid, IPC_RMID, 0);
    return 0;
}
```

〈실습〉

메시지 큐에서 다루었던 ex05-1.c 와 ex05-2.c 를 이용하여 다음 조건의 양방향 통신 프로그램 exercise-1.c과 exercise-2.c를 만들어 보시다

〈조건〉



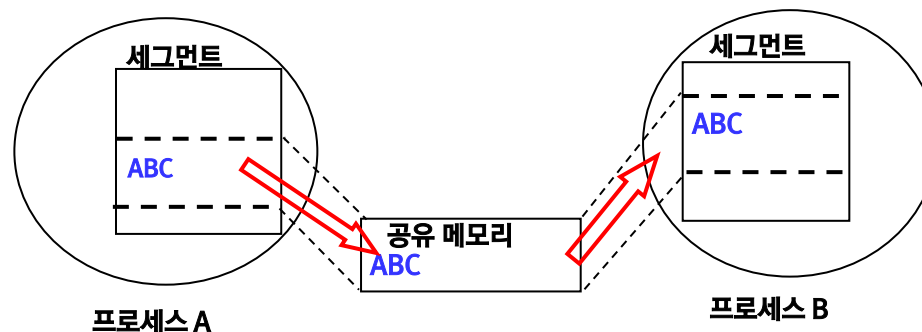
- 키보드로 입력 받은 문자열을 상대 프로세스로 전송
- 상대 프로세스가 보내는 문자열 출력
- 단, 문자열을 각각 한번에 하나씩 송수신한다.

공유 메모리

1. 공유메모리란?
2. 공유 메모리 관련 함수

공유 메모리

- 리눅스에서 프로세스란 서로 다른 공간에서 동시에 수행되는 실행의 흐름을 말한다. 프로세스가 다르면 메모리 공간도 다르다라는 의미이다. IPC 기능 중에는 프로세스간에 메모리를 공유할 수 있게 해주는 공유 메모리 기능이 제공된다. 이번 시간에는 이 기능에 대하여 알아보기로 하자.
- 공유 메모리 기능은 메모리의 일부 영역을 각 프로세스가 수행되는 가상의 주소 공간(Virtual Address Space)에 매핑 시켜주고, 이 주소에 접근하여 데이터를 읽어오거나 쓰기를 함으로써 통신하는 방법이다. 그림으로 표현하면 다음과 같다.



공유 메모리의 사용 예

공유 메모리 관련 함수

■ 공유 메모리 생성자 함수

- 공유 메모리 오브젝트를 생성하는 함수는 `shmget()`으로 그 원형은 다음과 같다.

```
#include <sys/shm.h>
#include <sys/ipc.h>

int shmget(key_t key, size_t size, int shmflg);
```

`msgget()` 함수에서와 마찬가지로 `shmget()` 함수에서도 동일한 키를 이용하는 프로세스와 메모리를 공유하게 된다.

첫 번째 인수 `key`에는 키 값을 지정한다.

두 번째 인수 `size`에는 공유할 메모리의 크기를 바이트 단위로 지정한다.

세 번째 인수 `shmflg`에는 `IPC_CREAT` 플래그와 사용 권한을 비트 OR 하여 지정한다. 동일한 키를 이용하는 공유 메모리 오브젝트가 이미 생성되어 있으면 `IPC_CREAT` 플래그는 무시되고 그 오브젝트의 ID 만 반환한다. 함수 호출이 성공하면 IPC ID를, 실패하면 `-1`을 반환한다.

- 만약 `IPC_CREAT`와 함께 `IPC_EXCL` 플래그가 사용되면 이미 지정한 키 값의 공유메모리가 존재할 경우에는 함수 호출은 실패하게 된다.

공유 메모리의 최대 크기

공유 메모리에 대한 리눅스 시스템의 제한 값이 얼마인지 확인하는 방법

```
root@ubuntu: ~  
root@ubuntu:~# ipcs -m -l  
  
----- Shared Memory Limits -----  
max number of segments = 4096  
max seg size (kbytes) = 32768  
max total shared memory (kbytes) = 8388608  
min seg size (bytes) = 1  
  
root@ubuntu:~#  
root@ubuntu:~# sysctl -a | grep kernel.shm  
kernel.shm_next_id = -1  
kernel.shm_rmid_forced = 0  
kernel.shmall = 2097152  
kernel.shmmax = 33554432  
kernel.shmmni = 4096  
root@ubuntu:~#
```

세마포와 메시지큐의 최대 크기

세마포에 관련된 제한 값을 확인하는 방법

```
root@ubuntu: ~  
root@ubuntu:~# ipcs -s -l  
  
----- Semaphore Limits -----  
max number of arrays = 128  
max semaphores per array = 250  
max semaphores system wide = 32000  
max ops per semop call = 32  
semaphore max value = 32767  
  
root@ubuntu:~#  
root@ubuntu:~# sysctl -a | grep kernel.sem  
kernel.sem = 250 32000 32 128  
kernel.sem_next_id = -1  
root@ubuntu:~#
```

메시지큐에 관련된 제한 값을 확인하는 방법

```
root@ubuntu: ~  
root@ubuntu:~# ipcs -q -l  
  
----- Messages Limits -----  
max queues system wide = 2263  
max size of message (bytes) = 8192  
default max size of queue (bytes) = 16384  
  
root@ubuntu:~#  
root@ubuntu:~# sysctl -a | grep kernel.msg  
kernel.msg_next_id = -1  
kernel.msgmax = 8192  
kernel.msgmnb = 16384  
kernel.msgmni = 2263  
root@ubuntu:~#
```

공유 메모리 관련 함수

■ 공유메모리 연결 및 해제 함수

- shmget() 함수에서 할당 받은 공유 메모리 세그먼트를 프로세스 공간으로 매핑해야 프로세스에서 이용할 수 있다. 또 프로세스에서 모든 사용이 끝나면 공유 메모리 공간을 프로세스로부터 분리해야 한다. 이 때 사용되는 함수가 shmat(), shmdt() 함수다. 다음은 그 함수의 원형이다.

```
void *shmat(int shm_id, const void *shm_addr, int shmflg);
```

```
int shmdt(const void *shmaddr);
```

shmat() 함수의 첫 번째 인수 shm_id는 shmget() 함수에서 반환 받은 값인 공유 메모리 오브젝트의 ID 를 전달한다.

두 번째 인수 shm_addr에는 할당된 공유 메모리를 프로세스에 매핑할 주소를 지정한다. 만약 NULL을 지정하면 시스템에서 주소를 정해 반환해준다. 사용자가 프로세스내의 주소를 모두 알 수 없기 때문에 대부분 NULL 값을 이용한다.

세 번째 인수 shmflg에는 0이나 SHM_RDONLY 등을 설정할 수 있다. SHM_RDONLY 은 읽기 전용의 공간으로 설정할 때 사용하고, 0은 읽기/쓰기가 모두 가능하게 설정할 때 사용한다.

공유 메모리 관련 함수

- 이 함수의 호출이 성공하면 메모리의 주소를 반환하고 그렇지 않으면 -1을 반환한다.
- `shmdt()` 함수는 `shmat()` 함수에서 받아온 주소를 인수로 전달하여 프로세스로부터 공유 메모리 공간을 분리시킨다. 이때 공유 메모리 공간이 완전히 삭제되는 것은 아니며 단지 프로세스에서 접근만 못하게 된다.

공유 메모리 관련 함수

■ 공유 메모리 삭제 및 제어 함수

- shmctl() 함수는 공유 메모리를 제어할 수 있는 함수로 다음과 같은 원형으로 이루어져 있다.

자료형에 대한 세부 사항은 다음 페이지 참조

```
int shmctl(int shm_id, int command, struct shmid_ds *buf);
```

첫 번째 인수 shm_id에는 shmget() 함수의 반환 값인 공유 메모리 오브젝트의 id 를 지정한다.

두 번째 인수 command 에는 공유 메모리 제어 명령을 지정한다. 공유메모리의 제어 명령은 메시지 큐의 제어 명령과 동일하다

제어 명령	의미
IPC_STAT	공유 메모리에 관련된 정보를 세번째 인수로 읽어온다. struct shmid_ds 구조체 이용
IPC_SET	공유 메모리에 관련된 정보를 세번째 인수에 전달한 값으로 설정한다. struct shmid_d s 구조체 이용
IPC_RMID	공유 메모리 객체를 삭제한다.
IPC_INFO	IPC 자원의 리눅스 설정 값을 읽어옴. shmctl 매뉴얼 참조

세 번째 인수 buf 에는 제어 명령에 따라 해당 구조체 포인터 변수를 지정한다.

이 함수의 호출이 성공하면 0을 반환하고, 실패하면 -1을 반환한다.

```
struct ipc_perm {
    key_t    __key; /* Key supplied to shmget(2) */
    uid_t    uid;   /* Effective UID of owner */
    gid_t    gid;   /* Effective GID of owner */
    uid_t    cuid;  /* Effective UID of creator */
    gid_t    cgid;  /* Effective GID of creator */
    unsigned short mode; /* Permissions + SHM_DEST and
                          SHM_LOCKED flags */
    unsigned short __seq; /* Sequence number */
};
```

```
struct shmid_ds {
    struct ipc_perm shm_perm; /* Ownership and permissions */
    size_t    shm_segsz; /* Size of segment (bytes) */
    time_t    shm_atime; /* Last attach time */
    time_t    shm_dtime; /* Last detach time */
    time_t    shm_ctime; /* Last change time */
    pid_t     shm_cpid; /* PID of creator */
    pid_t     shm_lpid; /* PID of last shmat(2)/shmdt(2) */
    shmatt_t   shm_nattch; /* No. of current attaches */
};
```

ex06.c는 공유 메모리 생성으로부터 삭제 과정을 ipcs 명령으로 추적한 프로그램이다

소스코드 (ex06.c)

```
if((shmid= shmget(0x123400, 30, 0660|IPC_CREAT))
== -1){
    perror("shmget");
    exit(1);
}
printf("\nAfter shmget...\n");
system("ipcs -m | grep 123400 ");
getchar();

if((shmaddr= shmat(shmid, (char *)0, 0)) == NULL){
    perror("shmat");
    exit(1);
}
printf("\nAfter shmat...\n");
system("ipcs -m | grep 123400 ");
getchar();

strcpy(shmaddr, "shared memory test");
printf("shmaddr = %p : %s\n", shmaddr, shmaddr);
sleep(1);
if(shmdt(shmaddr) == -1){ /* 프로세스와의 접속 끊기 */
    perror("shmdt");
    exit(1);
}
```

```
printf("\nAfter shmdt...\n");
system("ipcs -m | grep 123400 ");
getchar();

if(shmctl(shmid, IPC_RMID, (struct shmctl *)0) == -
1){
    perror("shmctl");
    exit(1);
}
printf("\nAfter shmctl(IPC_RMID)...\n");
system("ipcs -m | grep 123400")
```

ex07.c 는 공유 메모리에 관한 정보를 확인하는 프로그램이다.

소스코드 (ex07.c)

```
int shmid;
char *shmaddr;
struct shmid_ds shm_stat;

if((shmid= shmget(0x123400,30,0600|IPC_CREAT))== -1){
    perror("shmget");
    exit(1);
}
if((shmaddr= shmat(shmid,NULL,0))==NULL){
    perror("shmat");
    exit(1);
}
if(shmctl(shmid,IPC_STAT,&shm_stat)==-1){
    perror("shmctl");
    exit(1);
}
printf("세그먼트 크기 (bytes) : %d\n", (int) shm_stat.shm_segsz);
printf("최종 Attach 시간 : %s", ctime(&shm_stat.shm_atime));
printf("공유메모리 생성자(PID) : %d\n", shm_stat.shm_cpid);
printf("최근 사용했던 프로세스 PID : %d\n", shm_stat.shm_lpid);
printf("현재 Attach 되어있는 프로세스 수 : %d\n", (int)shm_stat.shm_nattch);
if(shmdt(shmaddr)==-1){
    perror("shmdt");
    exit(1);
}
if(shmctl(shmid,IPC_RMID,(struct shmid_ds *)0)==-1){
    perror("shmctl");
    exit(1);
}
```

공유 메모리를 이용하여 정보를 주고받는 통신 프로그램 ex08-1.c 와 ex08-2.c 을 분석하고 실행해봅시다.

소스코드 (ex08-1.c)	소스코드 (ex08-2.c)
<pre> int main() { int shmid, i, j; char *shmaddr; if((shmid=shmget(0x123400, 30, 0660 IPC_CREAT IPC_EXCL))==- 1){ if((shmid=shmget(0x123400, 30, 0660))==-1){ perror("shmget"); exit(1); } } if((shmaddr=shmat(shmid, (char *)0, 0))== NULL) { perror("shmat"); exit(1); } for(i=0; i<20; i++){ sprintf(shmaddr, "shared memory test %d", i+1); printf("send : %s\n", shmaddr); for(j=0; j<100000000; j++); } sprintf(shmaddr, "end"); if(shmdt(shmaddr)==-1) { perror("shmdt"); exit(1); } return 0; } </pre>	<pre> int main() { int shmid, i, j; char *shmaddr; if((shmid=shmget(0x123400, 30, 0660 IPC_CREAT IPC_EXCL))==-1) { if((shmid=shmget(0x123400, 30, 0660))==-1){ perror("shmget"); exit(1); } } if((shmaddr= shmat(shmid, (char *)0, 0))== NULL) { perror("shmat"); exit(1); } while(1) { if(!strcmp(shmaddr, "end")) break; if(!strcmp(shmaddr, "")) continue; printf("recv : %s\n", shmaddr); for(j=0; j<99999999; j++); } if(shmdt(shmaddr) == -1) { perror("shmdt"); exit(1); } if(shmctl(shmid, IPC_RMID, (struct shmid_ds *)0) == -1) { perror("shmctl"); exit(1); } return 0; } </pre>

세마포

1. 세마포란
2. 세마포 관련 함수

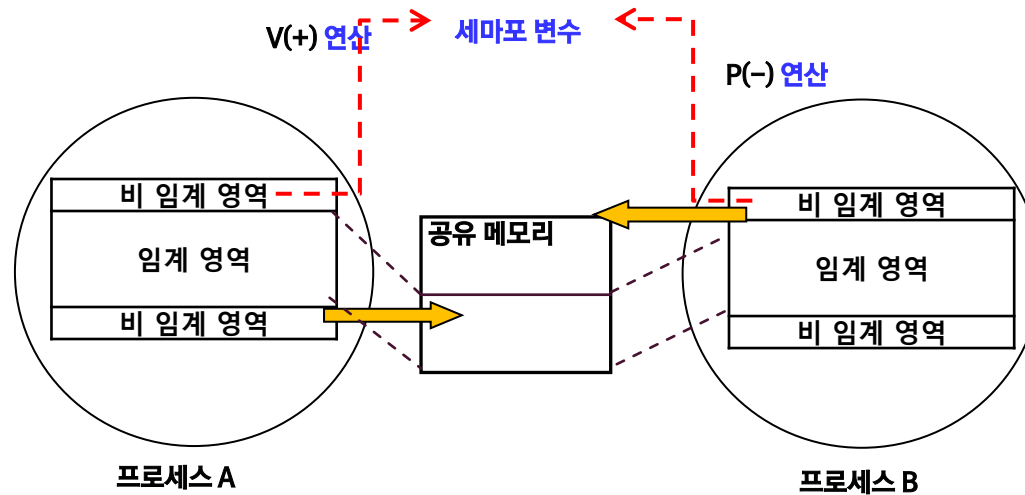
세마포란

- 일반적으로 프로세스간의 통신 시에 앞서 공유메모리의 예제에서 살펴보았듯이 동시에 같은 자원을 접근하는 문제가 발생한다. 여러 프로세스가 공유 자원들을 동시에 접근하는 경우의 문제를 해결하기 위해 리눅스에는 뮤텝스, 스핀락, 세마포 등 여러가지 메소드가 제공된다. 여기서는 일반적인 SystemV IPC 기능의 하나인 세마포를 통해 동기화 문제를 해결하고 스레드 관련 시간에 다른 동기화 방법을 살펴보기로 하자.
- 세마포란 두 가지 연산의 조합으로 구성된다.

P(세마포 변수) : 세마포 변수가 0 보다 크면 감소시키고, 0이라면 프로세스의 실행을 0보다 커질 때까지 중지한다.

V(세마포 변수) : 세마포 변수를 증가시켜 그 변수를 기다리면서 중지된 프로세스가 있으면 그 프로세스의 실행을 재개시킨다.
- 이 연산은 프로세스가 임계 영역으로 진입하기 전에 P 연산을 임계 영역을 지나면 V 연산을 해줌으로써 일종의 대문 역할을 수행한다. 다음 그림은 임계 영역과 세마포의 관계를 나타낸 것이다.

세마포란



두 프로세스간의 세마포 연산 예

- 세마포는 프로세스간 통신 보다는 통신시 일어날 수 있는 충돌 문제를 해결하기 위한 동기화 메소드로 사용된다. 세마포에는 연산의 개념이 있어 감소를 의미하는 P연산과 증가 연산을 의미하는 V연산으로 이루어진다. 한쪽 프로세스에서 임계 영역을 빠져나온 후 V 연산을 하면, 다른 프로세스에서 P 연산을 통해 임계 영역으로 진입이 가능하도록 함으로써 동기화가 이루어진다.

세마포 관련 함수

■ 세마포 생성자 함수

- 메시지 큐나 공유 메모리와 마찬가지로 세마포 오브젝트를 생성하는 함수는 `semget()`으로 다음과 같은 원형으로 이루어져있다.

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
```

```
int semget(key_t key, int num_sems, int sem_flags);
```

첫 번째 인수 `key`에는 같은 세마포를 이용하려는 프로세스끼리 동일한 값을 지정한다.

두 번째 인수 `num_sems`에는 세마포의 개수를 지정한다. 일반적으로는 1을 지정하나 공유할 데이터의 종류에 따라 각각을 따로 관리해야 하는 경우에는 그 개수를 지정한다.

세 번째 인수 `sem_flags`에는 `IPC_CREAT`와 함께 세마포의 접근 권한을 비트 OR하여 표현한다.

- 이 함수가 성공하면 세마포 오브젝트의 ID를 반환하고, 실패하면 `-1`을 반환한다. 만약 `IPC_CREAT`와 함께 `IPC_EXCL` 플래그가 사용되면 이미 지정한 키 값의 세마포가 존재할 경우에는 함수 호출은 실패하게 된다

세마포 관련 함수

■ 세마포 연산 함수

- `semop()` 함수는 세마포의 값을 변경하는데 사용하는 함수로 다음과 같은 원형으로 이루어져 있다.

```
int semop(int sem_id, struct sembuf*sem_ops, size_t num_sem_ops);

struct sembuf{
    short sem_num;
    short sem_op;
    short sem_flg;
};
```

첫 번째 인수 `sem_id` 는 `semget()` 함수에서 반환 받은 세마포 오브젝트의 ID 를 지정한다.

두 번째 인수 `sem_ops`는 구조체 자료형 `sembuf`의 멤버를 이용하여 표현한다. 예를 들어 세마포 변수 `sv`를 `semop()` 함수로 호출하면 다음과 같다.

```
struct sembuf sv = { 0, -1, SEM_UNDO };

semop(sem_id, &sv, 1);
```

```
struct sembuf sv = { 0, 1, SEM_UNDO };

semop(sem_id, &sv, 1);
```

세마포 관련 함수

```
struct sembuf sv = { 0, -1, SEM_UNDO };  
sv.sem_num=0;           // 세마포 번호  
sv.sem_op=-1;           // P 연산 (1 : V연산)  
sv.sem_flg=SEM_UNDO;    // 세마포 플래그  
semop(sem_id, &sv, 1);
```

여기에서 첫 번째 멤버 `sem_num` 에는 세마포 번호를 지정하고, 두 번째 멤버 `sem_op` 에는 P연산(-) 또는 V연산(+)을 지정한다

세 번째 멤버에는 일반적으로 `SEM_UNDO`를 설정한다. 이 플래그를 설정하면 현재 프로세스가 변경하는 세마포의 값을 운영체제가 추적할 수 있게 해주고, 프로세스에서 세마포를 해지하지 않고 종료하면 운영체제에서 알아서 세마포를 해지하도록 설정하는 플래그다. 이외에 설정 가능한 플래그에는 `IPC_NOWAIT` 플래그를 비트 OR 하여 표현할 수 있다.

- 세 번째 인수 `num_sem_ops`는 연산할 세마포의 개수를 지정한다. 이 함수가 성공하면 0을 반환하고, 실패하면 -1을 반환한다.

세마포 관련 함수

■ 세마포 제어 함수

- semctl() 함수 원형

```
int semctl(int sem_id, int sem_num, int command, .....);
```

첫 번째 인수 `sem_id`에는 `semget()` 함수의 반환 값인 세마포 오브젝트의 ID를 지정한다

두 번째 인수 `sem_num`는 `command` 인수에 지정되는 명령에 따라 세마포의 번호를 지정하거나 0을 지정한다

세 번째 인수 `command`에는 다음과 같은 세마포 제어 명령을 지정할 수 있다

command	의미
GETVAL	세마포의 현재 값을 얻는다.
GETPID	세마포에 접근했던 프로세스의 PID를 얻어온다.
GETNCNT	세마포의 값이 증가되기를 기다리는 프로세스의 개수를 얻어온다.
GETZCNT	세마포의 값이 0이 되기를 기다리는 프로세스의 개수를 얻어온다.
GETALL	세마포 집합의 모든 세마포에 대한 값을 얻어온다.
SETVAL	세마포의 값을 설정한다.
SETALL	세마포 집합의 모든 세마포에 대한 값을 설정한다.
IPC_STAT	세마포 오브젝트에 대한 정보를 얻어온다.
IPC_SET	세마포의 소유권과 사용 권한을 설정한다.
IPC_RMID	세마포 오브젝트를 삭제한다.

세마포 관련 함수

- command에 따라 네번째 인수의 자료형은 다음과 같은 **union semun** 공용체를 이용한다.

```
union semun {  
    int val;  
    struct semid_ds *buf;  
    unsigned short *array;  
    struct seminfo *__buf;  
}
```

각 자료형에 대한 세부 사항은 다음 페이지 참조

- 다음은 **semctl** 함수에 **SETVAL** 명령의 사용 예다.

```
union semun semctrl;  
  
semctrl.val = 1;  
semctl(sem_id, 0, SETVAL, semctrl);
```

위의 예는 세마포 0 번에 값을 1로 설정한다는 의미입니다. 이와 같은 표현은 주로 처음 세마포 객체 생성 후 초기값 설정시 이용한다.

```

struct semid_ds {
    struct ipc_perm sem_perm; /* Ownership and
permissions */
    time_t      sem_otime; /* Last semop(2) time
time_t      sem_ctime; /* Last change time
    unsigned long sem_nsems; /* Number of
semaphores in set */
};

```

```

struct ipc_perm {
    key_t      __key; /* Key supplied to semget(2)
    uid_t      uid; /* Effective UID of owner */
    gid_t      gid; /* Effective GID of owner */
    uid_t      cuid; /* Effective UID of creator */
    gid_t      cgid; /* Effective GID of creator */
    unsigned short mode; /* Permissions */
    unsigned short __seq; /* Sequence number */
};

```

```

struct seminfo {
    int semmap; /* Number of entries in semmap; unused within kernel */
    int semmni; /* Maximum number of semaphore sets in system */
    int semmns; /* Maximum number of semaphores in all semaphore sets */
    int semmnu; /* System-wide maximum number of undo structures; unused within kernel */
    int semmsl; /* Maximum number of semaphores in a set */
    int semopm; /* Maximum number of operations for semop(2) */
    int semume; /* Maximum number of undo entries per process; unused within kernel */
    int semusz; /* Size of struct sem_undo */
    int semvmx; /* Maximum semaphore value */
    int semaem; /* Max. value that can be recorded for semaphore adjustment (SEM_UNDO) */
};

```



공유 메모리에서 다루었던 실습 소스 중 ex08-1.c 와 ex08-2.c 는 실행시 충돌 현상이 일어났었다. 각각 exercise-3.c 과 exercise-4.c 을 수정하여 충돌 문제가 해결되었는지 확인해보자.

POSIX IPC

- 표준화에 의해서 탄생한 IPC system call
 - SUS 표준이지만 SysV IPC 보다 낮거나 비슷한 성능
 - linux에서는 컴파일시 rt library를 링킹해야함.(-lrt 옵션 사용)
 - 파일 인터페이스와 유사하게 설계
- mq_open()/shm_open()/sem_open()

SHARED MEMORY

- 가장 빠른 IPC 자원
 - 크리티컬 섹션에서의 데이터오염 주의
 - 락 메커니즘의 필요
- SysV SHM vs POSIX SHM
 - SysV SHM은 시스템내 특정 영역의 맵핑
 - POSIX는 가상 메모리 공간에 파일을 만드는 형식
 - POSIX SHM은 대응을 위해 mmap이 필요
 - Linux에서는 kernel 2.4 이후에 /dev/shm에 POSIX SHM공간을 생성

POSIX SHM

■ 사용함수

- shm_open(3) : 공유메모리의 파일기술자를 얻음(생성)
- mmap(2) : 공유메모리의 파일기술자를 메모리맵으로 맵핑.
- shm_unlink(3) : 공유메모리를 제거

■ 특징

- POSIX 계열은 저수준의 파일핸들링과 같은 인터페이스 제공
- 따라서 shm_open(3)은 마치 open(2)과 비슷한 형태를 가짐
- shm_unlink(3)도 unlink(2)와 흡사함

SHM_OPEN()

■ 함수 원형

```
int shm_open(const char *name, int oflag, mode_t mode);
```

- name : SHM을 생성할 파일경로명
Linux에서는 지정된 파일경로명은 /dev/shm을 root로 하여 생성됨
- oflag : 오픈시 사용할 플래그 (O_EXCL을 이용하는 방법을 가장 많이 사용)

O_RDONLY	읽기전용으로 오픈
O_RDWR	읽기/쓰기 가능으로 오픈
O_CREAT	해당 SHM이 존재하지 않은 경우 생성
O_EXCL	이미 SHM이 존재하는 경우 에러로 리턴: EEXIST
O_TRUNC	공유메모리가 이미 존재한다면 0 바이트로 줄임

- mode : 접근권한: open()함수의 mode와 동일

SHM_UNLINK(3)

- 함수 원형

```
int shm_unlink(const char *name);
```

- name : SHM의 파일경로명
Linux에서는 /dev/shm을 root로 하여 생성된 파일이 삭제됨
- POSIX SHM은 따로 detach하는 과정을 거치지 않음

MMAP (CON'T)

- 파일, 장치를 memory와 대응
 - SVR4 에서 파일과 프로세스의 1:n 대응 오버헤드의 해결책
 - 프로세스간의 데이터나 파일:메모리의 동기화로 사용
- 메모리맵 특징
 - 대응된 mmap은 포인터로 접근하므로 사용이 용이
 - 파일로 연결(attach)하면 메모리와 파일 사이의 동기화가 편리
 - 파일:메모리 동기화와 크리티컬 섹션 보호에 신경써야 함
 - mmap의 크기를 넘어서는 경우에 파일에는 영향을 주지 않음

MMAP (CON'T)

■ 사용 함수

mmap(2)	메모리와 파일의 대응 (mmap의 생성)
munmap(2)	파일과의 대응을 해제
msync(2)	메모리와 파일을 동기화
mprotect(2)	mmap 접근 권한 변경

- mmap은 파일을 먼저 열고 메모리에 동기화 하는 과정

■ 메모리맵의 두가지 방식

- shared
- private

MMAP()

■ mmap() 함수 원형

```
void * mmap(void *start, size_t length, int prot , int flags, int fd, off_t offset);
```

- start : 메모리 시작번지 (page 경계로...) : 주로 NULL 사용
- length : 대응시킬 mmap의 길이 (byte)
- prot : PROT_EXEC, PROT_READ, PROT_WRITE, PROT_NONE
- flags : MAP_FIXED, MAP_SHARED, MAP_PRIVATE
- fd : mmap을 만들 열려진 파일의 파일기술자
- offset : 대응시킬 파일의 위치 offset

MSYNC()

■ msync() 함수 원형

```
int msync(void *start, size_t length, int flags);
```

- start : 동기화시킬 주소
- length : 동기화시킬 mmap의 길이 (byte)
- flags

MS_ASYNC	msync 함수는 바로 리턴되고, 동기화는 이후 진행 즉 msync리턴시점에 동기화 보장할 수 없음
MS_SYNC	msync는 동기화를 모두 마친후 리턴
MS_INVALIDATE	메모리에 쓰여진 값을 무효로 하고, 파일상태로 갱신 (파일의 내용을 기준으로 롤백)

MUNMAP()

- munmap() 함수 원형

```
int munmap(void *start, size_t length);
```

- start : 동기화시킬 주소
- length : 동기화시킬 mmap의 길이 (byte)



posix_shm.c 를 통해 POSIX 공유메모리 함수의 사용법을 확인하고 SysV 공유 메모리 함수와 비교해 봅시다.

그리고 교재 5.3.4 POSIX IPC 함수(304페이지)에 소개된 POSIX 메시지 큐와 세마포 관련 함수를 살펴보고 예제를 실행하여 봅시다.

리눅스 시스템 프로그래밍

8장. 쓰레드



목차

1. 쓰레드란?
2. 쓰레드 생성과 종료
3. 쓰레드 통신
4. 쓰레드 동기화
5. 쓰레드의 클린업처리
6. 쓰레드의 시그널 처리

쓰레드란

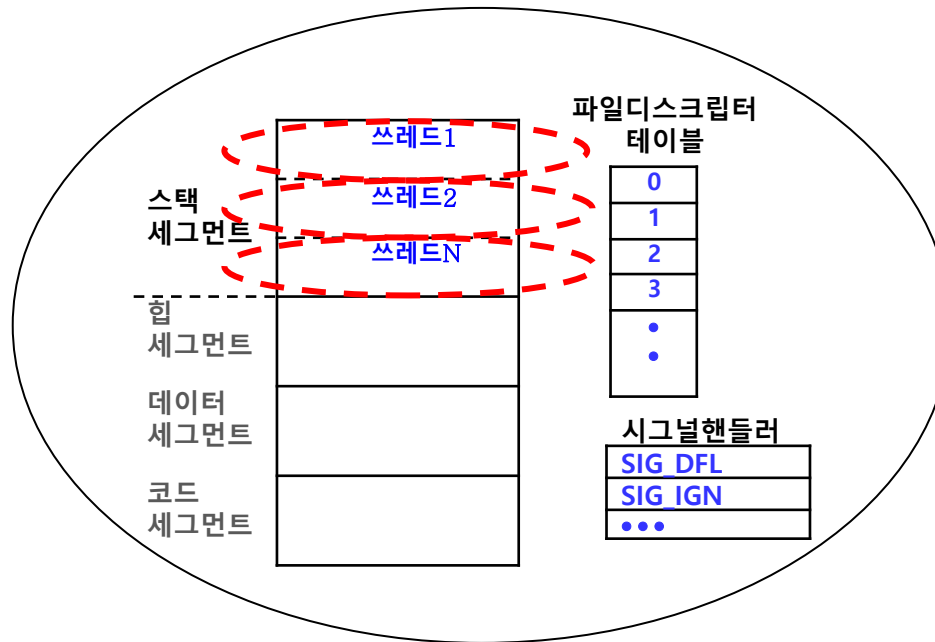
- 1.쓰레드 개념
- 2.쓰레드 관련 도구

쓰레드란

- 리눅스 및 유닉스에서 동시에 여러 작업을 수행시키려면 프로세스를 생성한다. 이 프로세스는 `fork()` 함수에 의해 생성되는데, 리눅스에서는 서로 공유할 수 있는 공간(코드 세그먼트)을 제외한 나머지 공간(데이터 세그먼트, 힙 세그먼트, 스택 세그먼트, 파일 디스크립터 테이블 등)을 새로운 공간으로 복사하는 방법으로 이루어진다. 따라서 새로운 프로세스를 만든다는 것은 시스템의 자원을 많이 소모하는 일이다. 프로세스의 특징인 동시 처리 기능을 만족하면서 시스템의 자원 소모를 최소화할 수 있는 방법이 쓰레드다. 이번 절에서는 쓰레드의 기본 개념과 구조 등을 살펴보기로 하겠다.
- 쓰레드 개념
 - 쓰레드란 한 프로세스 내에서 동작되는 여러 실행의 흐름으로, 프로세스내의 주소 공간이나 자원들을 대부분 공유하면서 실행된다. 새로운 프로세스를 생성시키지 않기 때문에 그만큼 자원을 아낄 수 있으며, 더 효율적으로 빠르게 움직일 수 있다. 또한 같은 프로세스 내에서 동작하므로 데이터를 공유하기가 쉽다는 장점도 가진다.

쓰레드란

- 다음은 쓰레드의 구조를 간단하게 그림으로 표현한 것이다.



프로세스와 쓰레드

- 그림에서 알 수 있듯이 쓰레드는 프로세스 내에서 각각의 스택 공간을 제외한 나머지 공간과 시스템 자원을 함께 공유하므로 프로세스를 이용하여 동시에 처리하던 일을 쓰레드로 구현한다면 메모리 공간은 물론 시스템 자원 소모도 현격히 줄어든 것이다

쓰레드란

- 이와 같이 프로세스를 생성하는 것 보다 쓰레드를 생성하는 것이 효율적이다. 특히 멀티 프로세서 환경에서는 더욱 효과가 탁월하다. 쓰레드간의 통신이 필요한 경우 별도의 자원을 이용하는 것이 아니라 전역 변수의 공간을 이용하여 데이터를 주고 받을 수 있다. 쓰레드의 장점을 정리하면 다음과 같다.
 - 시스템의 Throughput 이 향상된다.
 - 시스템의 자원 소모가 줄어든다.
 - 프로그램의 응답시간이 단축된다.
 - 프로세스간 통신 방법에 비해 쓰레드간의 통신 방법이 훨씬 간단하다.
- 쓰레드는 장점만 갖고 있는 것이 아니라 다음과 같은 단점도 지니고 있다.
 - 멀티 쓰레드 프로그램을 작성하는 경우에는 주의 깊게 설계해야 한다. 한 쓰레드에서 발생한 문제가 다른 쓰레드에도 영향을 미칠 수 있다. 예를 들어 한 쓰레드에서 잘못된 메모리 영역을 침범하여 프로세스가 죽는 경우 이 프로세스에서 생성된 모든 쓰레드도 함께 죽어버린다. 물론 이런 일이 생기지 않도록 프로그램으로 보완해야 한다.
 - 프로그램 디버깅이 어렵다.

쓰레드란

- 쓰레드는 POSIX 위원회가 표준을 정하기 이전에는 각 운영체제 벤더에서 각각의 구조를 갖고 구현되어 왔다. POSIX 1003.1c 쓰레드 표준안이 마련이 되면서 비로소 동일한 방법으로 쓰레드를 구현할 수 있게 되었다 리눅스의 쓰레드도 이 표준안을 거의 수용하고 있어 공통된 POSIX 쓰레드 라이브러리 함수들을 통해 쓰레드 구현이 가능하다.

쓰레드 관련 도구

- 리눅스에서 쓰레드를 확인하거나 제어할 수 있는 명령이나 도구는 그리 많지 않다.

- 쓰레드 수행 확인

`ps -eLf | grep 프로세스명`

- 다음은 rsyslogd에 대한 프로세스 상태를 확인하는 명령의 수행 결과다.

```
user@ubuntu:~$ ps -eLf | grep rsyslog | grep -v grep
syslog      677      1   677   0    4 Dec16 ?        00:00:00 rsyslogd
syslog      677      1   678   0    4 Dec16 ?        00:00:00 rsyslogd
syslog      677      1   679   0    4 Dec16 ?        00:00:00 rsyslogd
syslog      677      1   680   0    4 Dec16 ?        00:00:00 rsyslogd
user@ubuntu:~$
```

프로세스와 쓰레드 상태 보기

쓰레드 생성과 종료

1. 쓰레드 프로그램 컴파일
2. 쓰레드 생성과 종료 함수

쓰레드 프로그램 컴파일

- 리눅스에서 지원되는 쓰레드는 POSIX에서 표준으로 제안한 thread 라이브러리로 함수 명들은 대부분 pthread로 시작한다. 이 함수들을 이용하여 프로그램을 작성한 후 컴파일하려면 다음과 같은 옵션을 함께 컴파일 해야 한다.

gcc 소스파일명 -o 실행파일명 -D_REENTRANT -lpthread

- 여기에서 -D_REENTRANT 옵션은 C 소스 프로그램에 각 쓰레드를 위한 고유한 errno와 스택 공간을 지원하는 코드가 포함되도록 하는 지시자 이다. 만약 이 옵션을 설정하지 않는다면 각 함수의 실행이 실패한 경우 설정되는 errno 변수의 값이 여러 쓰레드에 의해 동시에 접근이 될 수 있어 다른 쓰레드의 결과를 참조하는 경우도 발생할 수 있다. 이런 경우를 막기 위해 errno 변수 이외에 재진입 가능한 함수로 변경되거나 쓰레드 별 스택을 별도로 할당하는 작업들을 처리한다. -lpthread 옵션은 라이브러리 함수들을 링크하기 위한 옵션이다.
- 컴파일할 때마다 매번 컴파일 옵션을 직접 입력해도 좋으나 다음과 같이 Makefile 을 작성해 놓으면 좀더 쉽게 컴파일 할 수 있다. Makefile 파일의 내용과 사용법은 다음과 같다.

```
user@ubuntu: ~/online_lsp/ch15
user@ubuntu:~/online_lsp/ch15$ cat Makefile
CFLAGS=-D_REENTRANT
LDLIBS=-lpthread
user@ubuntu:~/online_lsp/ch15$ ls sample.c
sample.c
user@ubuntu:~/online_lsp/ch15$ make sample
cc -D_REENTRANT sample.c -lpthread -o sample
user@ubuntu:~/online_lsp/ch15$
```

쓰레드 생성과 종료 함수

■ 쓰레드 생성자 함수

- 쓰레드를 생성하는 함수는 pthread_create() 로, 함수의 원형은 다음과 같다.

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *thread, pthread_attr_t *attr,  
                  void * (*start_routine(void *)), void *arg);
```

여기에서 첫번째 인수에는 쓰레드가 생성되면 그 쓰레드 ID를 받아올 변수의 주소를,

두번째 인수에는 쓰레드의 속성을 설정하는 값을 전달한다. 이 속성에 대해서는 다음 절에서 살펴보기로 하고 이 절에서는 NULL을 설정하기로 한다.

세번째 인수에는 쓰레드가 생성되면서 수행할 코드가 들어있는 함수의 이름을,

네번째 인수에는 그 함수로 전달할 인수 값을 설정한다.

쓰레드 생성이 성공하면 0을 반환하고 실패하면 에러 코드를 반환한다.

```
void *thread_routine(void)  
{  
    ...  
}
```

```
pthread_create(&t_id, NULL, thread_routine, NULL);
```

쓰레드 생성과 종료 함수

■ 쓰레드 종료를 기다리는 함수

- pthread_join() 함수는 프로세스의 wait() 함수와 같이 쓰레드의 종료를 기다리는 함수로 다음과 같은 원형을 가지고 있다.

```
#include <pthread.h>
```

```
void pthread_join(pthread_t thread, void **thread_return);
```

- 여기에서 첫 번째 인수 thread에는 종료를 기다리는 쓰레드의 ID, 즉 pthread_create() 함수에서 받아온 값을 지정하고, 두 번째 인수에는 쓰레드 종료 시 반환하는 값을 가리키는 포인터의 포인터를 전달한다.

쓰레드 생성과 종료 함수

■ 쓰레드 종료 함수

- pthread_exit() 함수는 쓰레드를 종료시킬 때 호출하는 함수로 그 원형은 다음과 같다.

```
#include <pthread.h>

void pthread_exit(void *retval);
```

이 함수는 마치 프로세스가 종료할 때 exit() 함수를 호출하듯이 쓰레드 종료 시 호출하는 함수로 retval에 반환 값의 포인터를 전달한다. 이때 지역 변수의 주소를 실어주면 안된다. 왜냐하면 쓰레드가 종료되면 지역변수의 공간인 스택도 반납하므로 참조할 수 없기 때문이다. 이 포인터는 pthread_join() 함수의 두번째 인자로 전달된다.

- 만약 pthread_cleanup_push() 가 정의되어 있다면, pthread_exit 가 호출될 경우 cleanup handler 가 호출된다. 보통 이 cleanup handler 은 메모리를 정리하는 등의 일을 하게 된다



ex01.c 를 열어 리눅스에서 쓰레드 생성과 종료에 관한 부분을 완성하고 실행해보자.

쓰레드 생성과 종료 함수

■ 쓰레드 종료 요청 함수

- 프로세스에서 kill() 을 이용하여 다른 프로세스로 시그널을 전송할 수 있듯이 쓰레드에서도 다른 쓰레드로 종료를 요청할 수 있다. 그리고 종료 요청을 받은 쓰레드는 동작 여부를 설정할 수도 있다. 이런 기능을 가진 함수를 살펴보자.

```
#include <pthread.h>
```

```
int pthread_cancel(pthread_t thread);
```

여기에서 thread는 종료 요청을 보낼 쓰레드의 ID를 나타낸다. 이 요청을 받은 쓰레드에서는 다음 함수를 이용하여 동작 여부를 설정할 수 있다.

```
#include <pthread.h>
```

```
int pthread_setcancelstate(int state, int *oldstate);
```

여기에서 첫 번째 인수인 state 에는 다음과 같은 값을 설정할 수 있다.

PTHREAD_CANCEL_ENABLE : 종료 요청 허용

PTHREAD_CANCEL_DISABLE : 종료 요청 무시

두 번째 인수 oldstate에는 이전 상태를 얻어올 수 있다.



ex02.c 를 열어 쓰레드 강제 종료 방법을 확인하고 실행해보자. 종료 요청시 쓰레드에서 허용 유무가 가능한지도 알아보자

쓰레드 생성과 종료 함수

■ 쓰레드 ID 확인 함수

- 현재 수행되고 있는 쓰레드의 ID 를 확인하는 함수는 pthread_self로 함수의 원형은 다음과 같다.

```
#include <pthread.h>
```

```
pthread_t pthread_self(void);
```

- pthread_self() 함수를 호출한 쓰레드의 ID를 반환한다.

■ 쓰레드 분리함수

- 쓰레드의 기본 속성은 joinable 속성으로 이 속성으로 생성된 쓰레드는 main 쓰레드에서 pthread_join() 함수를 호출해야 쓰레드가 사용하던 자원들이 반납된다. 그런데 이 pthread_join() 함수를 호출하면 main 쓰레드는 블록 되어지기 때문에 다른 작업을 수행할 수 없다. 예를 들어 쓰레드를 수행중인 상태에 새로운 이벤트가 발생되어 쓰레드를 새로 생성해야 되는 경우 문제가 된다. 이런 경우에는 pthread_detach() 함수를 이용하여 해결할 수 있다.

쓰레드 생성과 종료 함수

```
#include <pthread.h>
```

```
int pthread_detach(pthread_t thread);
```

- pthread_detach()함수는 쓰레드 ID가 thread인 쓰레드를 main 쓰레드에서 분리시키는 함수로,
- 해당 쓰레드가 종료되는 즉시 쓰레드의 모든 자원을 반납(free)한다는 의미이다. detach상태가 아닐 경우 쓰레드가 종료한다고 하더라도 pthread_join()을 호출하지 않는 한 자원을 되돌려주지 않는다.

쓰레드가 detach상태로 되어있는 경우 해당 쓰레드에 대하여 pthread_join() 함수를 호출하면 실패한다. 성공하면 0을, 실패하면 0이 아닌 값을 반환한다.

ex03.c 를 통해 detach 속성과 joinable 속성에 대해 살펴보자. 우선 detach 속성으로 테스트하여 결과를 확인하고 소스코드에서 pause() 함수를 주석 처리한 후 pthread_join() 함수를 호출해보자

쓰레드간의 통신

- 쓰레드 개념

앞서 쓰레드의 개념과 구조를 살펴본 바에 의하면 쓰레드란 프로세스 내의 자원들을 최대한 공유하는 작업 단위였다. 쓰레드 간에 공유하는 공간 중 하나인 데이터 세그먼트는 전역변수나 static 변수가 할당되는 공간으로 전역 변수를 선언하면 쓰레드간에 공유 메모리 공간으로 사용할 수 있다는 의미이다. ex04.c를 통해 그 의미를 확인해보자.



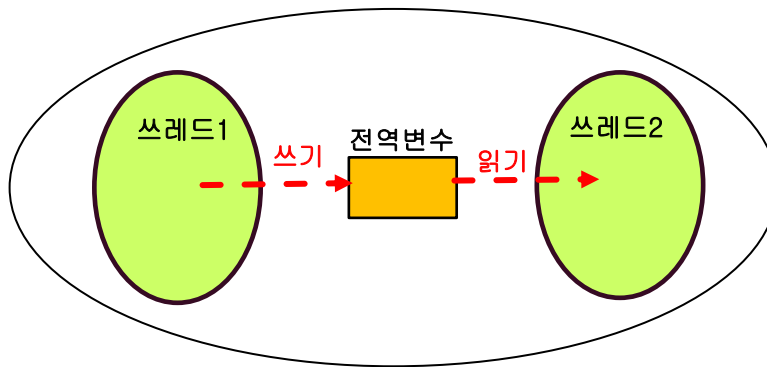
이제 쓰레드 두 개를 만들어 서로 통신하는 프로그램인 ex05.c를 분석하고 실행해보자

쓰레드 동기화

1. 동기화
2. 뮤텝스
3. 세마포

동기화

- 멀티쓰레드 환경에서 쓰레드간의 통신이 이루어질 때 동기화는 반드시 함께 고려해야 할 요소다.
- 다음과 같이 가정해보자



- 그림은 동일한 전역변수에 대해 쓰레드1에서는 쓰기 작업을, 쓰레드 2에서는 읽기 작업을 수행하는 경우의 예이다. 이 경우, 쓰레드1이 쓰기 작업을 수행하고 있는 도중에 쓰레드2에서 전역변수에 있는 값을 읽어간다면 잘못된 값을 읽어갈 확률이 높다. 그러므로 쓰레드간 통신이 필요할 때는 하나 이상의 쓰레드가 동시에 동일한 자원에 접근하지 못하도록 동기화 기법을 이용하여 통제를 해야 한다.

동기화

- 리눅스에서 제공하는 스레드 통신의 동기화 기법에는 다음과 같은 종류가 있다. 각 메소드의 특징과 함께 살펴보자.

동기화 메소드	특징
뮤텍스	락킹(locking)알고리즘을 이용하여 특정 시간에 공유 자원에 접근을 한 스레드로 제한하는 동기화 메소드
조건변수	공유 자원의 특정 조건에 따라 스레드의 실행을 멈추거나 다시 실행을 재개하도록 제어하는 동기화 메소드
세마포	세마포 연산을 성공하는 경우(-1 연산 결과가 0보다 크거나 같은 경우)에만 공유 자원에 접근이 가능하도록 통제하는 동기화 메소드
rwlock	읽기 락(read lock) 과 쓰기 락(write lock) 을 구분해두고 공유 자원에 대한 읽기 작업 시에는 여러 스레드의 접근을 허용하고, 쓰기 작업 시에만 독점적으로 공유 자원의 접근을 허용하는 동기화 메소드
스핀락	짧은 시간의 락이 필요한 구간에 스핀 락을 이용하여 문맥교환을 지연시키고 CPU 사용 시간을 늘려 공유 자원에 접근을 수행하는 동기화 메소드
배리어	특정 작업을 수행하는 스레드간에 어떤 조건에 이르렀을 때 함께 작업을 수행하도록 동기화하는 메소드

동기화

- 어떤 동기화 메소드를 선택할 것인지는 쓰레드간의 통신 방식에 따라 다를 것이다. 예를 들어 모든 쓰레드가 공유 자원에 대해 읽기 쓰기 동작을 모두 수행하는 경우라면 한 순간에 반드시 하나의 쓰레드만 공유 자원에 접근해야 하므로 뮤텝스가 적절한 메소드라 할 수 있다. 반면 앞의 그림과 같이 생산자와 소비자 구조의 쓰레드 구조라면 세마포나 조건 변수가 적절할 것이다. 또 쓰레드 구조상 읽기 작업이 주로 많이 일어난다면 rwlock를 선택하는 것이 좋을 것이다. (물론 좀더 심도있는 분석에 따라 메소드 선택을 해야한다.) 쓰레드의 설계의 특성에 맞추어 적절한 동기화 메소드를 선택해야 효율적인 멀티 쓰레드 프로그램을 만들 수 있다
- 앞서 소개된 메소드 중 가장 널리 사용되는 뮤텝스와 세마포의 사용방법을 살펴보자.

무텍스

- 무텍스란 코드의 임계 영역을 한 순간 특정 스레드 하나만 사용할 수 있도록 임계영역에 진입할 때마다 무텍스를 lock하고, 임계영역을 빠져 나올 때마다 무텍스를 release 하는 방법으로 동기화를 시키는 메소드를 말한다.
- 무텍스 관련 함수
- 스레드들 간의 동기화를 시킬 수 있는 방법 중에 가장 쉽고 간단한 방법이 무텍스를 이용하는 방법이다. 무텍스 관련 함수는 다음과 같다.

```
#include <pthread.h>
```

```
int pthread_mutex_init(pthread_mutex_t *mutex,  
const pthread_mutexattr_t *mutexattr);  
int pthread_mutex_lock(pthread_mutex_t *mutex);  
int pthread_mutex_unlock(pthread_mutex_t *mutex);  
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- 이제 각 함수에 대한 세부 내용을 살펴보자.

mutex

■ mutex 생성 및 초기화

```
int pthread_mutex_init(pthread_mutex_t *mutex,  
const pthread_mutexattr_t *mutexattr);
```

- pthread_mutex_init() 함수는 mutex를 초기화하는 함수로 첫번째 인수에 mutex 객체의 ID를 받아오고, 두 번째 인수에는 mutex의 속성을 설정할 수 있다. mutex의 속성에는 다음과 같은 세 종류의 유형이 있다.

mutex 타입	중복 잠금시	성능	용도
PTHREAD_MUTEX_NORMAL	데드락 상태	빠름	기본 mutex로 사용
PTHREAD_MUTEX_ERRORCHECK	오류 반환	약간 느림	오류 검사용으로 사용
PTHREAD_MUTEX_RECURSIVE	중복 잠금 허용	약간 느림	중복 잠금을 허용하는 경우 사용

mutex

- mutex 변수의 초기화 방법은 다음과 같다.

예1) `pthread_mutex_t mutexid=PTHREAD_MUTEX_INITIALIZER;`

mutex 선언과 동시에 PTHREAD_MUTEX_INITIALIZER 매크로를 이용하여 정적으로 초기화하는 방법이다.

예2) `pthread_mutex_t mutexid;`
`pthread_mutex_init(&mutexid, NULL);`

mutex 변수 mutexid를 선언하고 기본 속성으로 초기화하는 방법이다. 리눅스에서 mutex 속성의 기본은 PTHREAD_MUTEX_NORMAL 이다.

무텍스

■ 무텍스 잠금 및 해제

```
int pthread_mutex_lock(pthread_mutex_t *mutex);  
int pthread_mutex_trylock(pthread_mutex_t *mutex);  
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

pthread_mutex_lock()과 pthread_mutex_unlock() 함수는 임계 영역으로 진입하기 전에 무텍스를 잠궈 다른 스레드가 진입하지 못하도록 설정하고, 임계 영역을 빠져 나올 때 무텍스를 해제하는 함수다. 만약 이 무텍스를 다른 스레드가 이미 잠궈놓은 상태라면 스레드는 수행을 멈추고 무텍스가 해제될 때까지 기다린다.

pthread_mutex_trylock() 함수는 pthread_mutex_lock() 함수와 동일한 기능을 수행하되 다른 스레드에 의해 무텍스가 잠겨있는 경우에 함수 안에서 블록되지 않고 즉시 반환한다. 이때 반환 값은 EBUSY 가 된다.

mutex

- mutex 잠금 및 해제

- mutex 잠금의 사용 예

```
if((ret_value=pthread_mutex_lock(&mutexid))!=0) // NORMAL Type
    fprintf(stderr, "errno : %d\n", ret_value);
```

mutex 잠금을 시도해서 성공하면 반환하고 실패하면 pthread_mutex_lock() 함수에서 블록된다.

- mutex 잠금을 해제하는 사용 예

```
if((ret_value=pthread_mutex_unlock(&mutexid))!=0)
    fprintf(stderr, "errno : %d\n", ret_value);
```

무텍스

- 무텍스 소멸

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

pthread_mutex_destroy() 함수는 무텍스 객체를 소멸시키는 함수다.

- 무텍스 사용

앞서 쓰레드간 통신시 충돌문제가 발생한 ex05.c를 무텍스를 이용하여 해결해보자.



ex05.c 의 쓰레드 충돌 문제를 뮤텁스를 이용하여 해결해보자.

세마포

- 세마포란 공유하는 데이터에 대한 파수꾼 역할을 하는 메소드로 임계 영역을 진입할 때마다 세마포의 값을 확인하여 세마포를 얻을 수 없을 때에는 임계영역에 들어갈 수 없고, 오로지 세마포를 얻을 수 있을 때에만 세마포의 값을 줄여주고 임계 영역으로 들어갈 수 있다. 또한 쓰레드에서 공유하는 데이터를 쌓아줄 때마다 세마포의 값을 늘려줌으로써 세마포의 값이 공유 데이터의 개수와 일치하도록 하는 연산 방법으로 구현된다.
- 세마포 관련 함수

```
#include <semaphore.h>
```

```
int sem_init(sem_t *sem, int pshared, unsigned int value);  
int sem_wait(sem_t *sem);  
int sem_trywait(sem_t *sem);  
int sem_post(sem_t *sem);  
int sem_destroy(sem_t *sem);
```

세마포

■ 세마포 초기화

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

- `sem_init()` 함수는 세마포 객체를 초기화하는 함수로 첫 번째 인수 `sem`에는 객체의 ID를 받아올 변수의 주소를 전달하고, 두 번째 인수 `pshared`에는 세마포 공유 타입을 지정한다. 공유 타입에는 한 프로세스내의 쓰레드간에 세마포를 이용하는 경우에는 0을, 다른 프로세스와 세마포를 함께 이용하는 경우에는 0이 아닌 값을 지정한다. 세 번째 인수 `value`에는 세마포의 초기 값을 지정한다.
- `sem_init()` 함수의 사용 예를 살펴보자

```
예) if( sem_init(&semid, 0, 1)!=0)  
    fprintf(stderr, "semid =%d is fail\n" );
```

세마포 객체를 같은 프로세스 내의 쓰레드 간에 이용 가능하도록 설정하고, 초기값은 1로 초기화한다.

세마포

■ 세마포 연산

```
int sem_wait(sem_t *sem);  
int sem_trywait(sem_t *sem);  
int sem_post(sem_t *sem);
```

- `sem_wait()` 함수는 세마포의 값을 1 감소, `sem_wait()` 함수는 세마포의 값을 1 증가 시키는 함수로, `sem_wait()` 함수 호출 시 세마포의 값이 0 이면 다른 스레드에서 `sem_post()` 함수에 의해 세마포의 값을 증가시킬 때까지 기다린다.
- `sem_trywait()` 함수는 `sem_wait()` 함수와 기능은 동일하나 연산이 실패하더라도 즉시 반환된다. 이때 `errno` 변수에 `EAGAIN` 이 설정된다.

세마포

예1) `if(sem_wait(&semid)==-1)
 perror("sem_wait");`

세마포 값에 -1 연산을 시도하여 결과가 음수이면 이 함수 안에서 블록 된다. 다른 쓰레드에서 이 세마포에 +1 연산을 시도할 때까지!!

예2) `while(1) {
 if(sem_trywait(&semid)==-1)
 if(errno==EAGAIN)
 continue;
 else
 break;
}`

세마포 값에 -1 연산을 시도 한 후 즉시 이 함수를 빠져 나온다. 반환 값이 -1인 경우 실패한 경우인데 `errno` 변수가 `EAGAIN` 인 경우는 연산을 실패한 경우이므로 다음 반복을 진행하고 다른 경우에는 반복문을 빠져나온다,

세마포

- 세마포 소멸

```
int sem_destroy(sem_t *sem);
```

이 함수는 세마포를 삭제하는 함수다

- ex05.c 에서 발생한 충돌 문제를 이번에는 세마포를 이용해 해결해봅시다.

ex05.c의 쓰레드 간 충돌 문제를 세마포를 이용하여 동기화를 구현해보자

소스코드 (ex07.c)

```
#define LOOP_MAX 10
```

```
int commonCounter= 0;
```

```
sem_t semid;
```

```
void * inc_thread(void *);
```

```
int main(void)
```

```
{  
    pthread_t tid1;  
    pthread_t tid2;
```

```
    if((pthread_create( &tid1, NULL, inc_thread, "thread1")) ||  
        (pthread_create( &tid2, NULL, inc_thread, "thread2"))){  
        perror("pthread_create");  
        exit (errno);  
    }
```

```
    pthread_join(tid1, (void **)NULL);  
    pthread_join(tid2, (void **)NULL);  
    sem_destroy(&semid);  
    return 0;
```

```
}
```

```
void* inc_thread(void *arg)
```

```
{
```

```
    int loopCount;
```

```
    int temp;
```

```
    char buffer[80];
```

```
    int i;
```

```
    int ret_value;
```

```
    for (loopCount = 0; loopCount < LOOP_MAX; loopCount++){
```

```
        sprintf (buffer, "<%s> Common counter : from %d to ",  
                (char*) arg, commonCounter);  
        write(1, buffer, strlen(buffer));
```

```
        temp = commonCounter;  
        for(i=0; i<900000; i++);    // for delay  
        commonCounter = temp + 1;  
        sprintf (buffer, "%d\n", commonCounter);  
        write(1, buffer, strlen(buffer));
```

```
    }  
    for(i=0; i<500000; i++);    // for delay  
}
```

쓰레드의 클린업 처리

■ 클린업 처리의 필요성

- 쓰레드 프로그램 수행 중에 뮅텍스 잠금 상태에서 또는 동적 메모리를 할당 받아 사용중인 상태에서 `pthread_cancel()` 함수에 의해 강제 종료 요청을 받아 바로 종료한다면 어떤 일이 발생할까?
- 해당 뮅텍스를 이용하는 모든 쓰레드가 데드락 상태에 빠지게 되거나 할당 받았던 메모리를 반납하지 못하므로 메모리 누수의 원인이 된다. 이런 문제가 일어나지 않도록 대비책을 세워야 하는데 그 방법이 클린업 함수를 이용하는 것이다. 이제 클린업 함수의 종류와 그 사용법을 익혀보자

■ 클린업 함수

- 클린업 함수란 쓰레드 종료시 수행해야 할 핸들러 함수를 등록하는 함수로 안전하고 신뢰성있는 쓰레드 프로그램 작성시 반드시 구현해야 할 함수다. 함수의 원형은 다음과 같다.

쓰레드의 클린업 처리

```
#include <pthread.h>
```

```
void pthread_cleanup_push(void (*routine)(void *), void *arg);  
void pthread_cleanup_pop(int execute);
```

- 여기서 pthread_cleanup_push 함수는 핸들러를 등록하는 함수로 첫번째 인자에는 핸들러 함수의 이름을, 두번째 인자에는 핸들러 함수 수행 시 인자로 전달할 내용을 지정합니다. pthread_cleanup_pop 함수는 pthread_cleanup_push 함수로 등록했던 핸들러 함수를 삭제하는 함수로 등록한 순서의 역순, 즉 스택과 같은 순서로 삭제된다. 이 함수 호출시 인자로 0을 전달하면 핸들러 함수를 수행하지 않고 삭제하고, 1을 주는 경우에는 핸들러 함수를 수행한 후 삭제한다

ex08.c는 클린업 함수를 이용한 예제이다 소스를 분석하고 실행해보자

소스코드 (ex08.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>

void cleanup(void *arg)
{
    printf("%s called.\n", (char *)arg);
    free(arg);
    printf("cleanup : free called\n");
}

void * thread_function(void *arg)
{
    int length=strlen((char*)arg);
    char* ptr=(char*) malloc(length+strlen("_handler")+1);

    printf("thread_function start\n");
    strcpy(ptr, arg);
    strcat(ptr, "_handler");
    pthread_cleanup_push(cleanup, ptr);
    sleep(5);
    pthread_cleanup_pop(0);
    free(ptr);
    printf("thread_function : free called\n");
    return NULL;
}
```

```
int main(int argc, char* argv[])
{
    int ret;
    pthread_t tid;
    char* value;

    printf("main start\n");
    if(argc!=2) {
        fprintf(stderr, "Usage : %s [thread | cleanup]\n",
argv[0]);
        exit(1);
    }
    value=argv[1];
    if((ret=pthread_create(&tid, NULL, thread_function,
(void *)value))!=0) {
        perror("pthread_create1");
        exit(1);
    }
    if(strcmp(value, "cleanup")==0) {
        sleep(2);
        pthread_cancel(tid);
    }
    pthread_join(tid, NULL);
    printf("main end\n");
    return 0;
}
```

쓰레드의 클린업 처리

- 앞서 살펴본 예제 ex08.c 에서 처럼 쓰레드 프로그램이 언제나 안정적으로 수행된다면 클린업 함수를 구현할 필요가 없겠으나 프로그램에서 예외란 언제 어디서든 발생할 수 있으며 이로 인해 쓰레드가 갑자기 종료되는 경우 다른 쓰레드에 영향을 미칠 수 있으므로 신뢰성 있는 프로그램 개발을 위해 클린업 함수 사용을 잘 기억해 두어야 한다.

쓰레드와 시그널

- 멀티쓰레드 환경에서의 시그널
- 리눅스의 시그널은 원래 프로세스 기반으로 설계된 자원으로 한 프로세스에서 다른 프로세스로 비동기적으로 발생하는 이벤트를 알리기 위한 수단이다. 반면 멀티 쓰레드 환경에서의 시그널은 프로세스와는 달리 어떤 쓰레드에서 처리할지 예측이 불가능하다. 따라서 쓰레드 프로그램에서는 별도의 시그널 처리가 필요하다.
- 일반적으로 쓰레드 프로그램에서 시그널 처리는 다음과 같은 방법으로 구현된다.
 - 각 쓰레드별로 시그널 블록 마스크를 다르게 설정하는 방법
 - 시그널 전담 처리 쓰레드를 운영하는 방법
- 시그널 처리 목적에 따라 적절한 방법을 선택하여 구현하도록 한다.
- 이제 쓰레드에서의 시그널 처리를 위해 필요한 함수를 알아보도록 하자.

쓰레드와 시그널

- 쓰레드에서의 시그널 처리 함수
- 시그널 블록 마스크 설정 함수

```
#include <pthread.h>
#include <signal.h>
```

```
int pthread_sigmask(int how, const sigset_t *newmask, sigset_t *oldmask);
```

- 여기서 첫번째 인자 how에는 SIG_BLOCK, SIG_UNBLOCK, SIG_SETMASK 중 하나로 설정하는데 SIG_BLOCK은 현재 설정된 시그널 마스크에 newmask를 추가하고, SIG_UNBLOCK는 현재 설정된 시그널 마스크에서 newmask를 삭제(해제)하게 된다. 그리고 SIG_SETMASK는 현재 시그널 마스크를 newmask로 변경하게 된다. 세번째 인자 oldmask에는 기존의 시그널 마스크를 받아올 수 있다
- 이 함수는 앞서 시그널 관련 장에서 학습했던 sigprocmask()와 유사한 함수로 시그널 마스크가 프로세스가 아니라 쓰레드 단위로 설정된다는 차이가 있다. sigprocmask() 함수는 쓰레드에 안전한 함수가 아니므로 쓰레드 프로그램에서는 반드시 pthread_sigmask() 함수를 이용해 시그널 마스크를 설정해야 한다.

쓰레드와 시그널

```
sigset_t sigset, oldsigset;  
sigfillset(&sigset);  
pthread_sigmask(SIG_SETMASK, &sigset, &oldsigset);
```

- 위의 예는 쓰레드의 시그널 마스크를 모든 시그널을 블록하도록 설정하는 부분이다. 이 부분을 수행하면 쓰레드에서는 시그널을 모두 차단하므로 사용자의 키보드에서 Control C 나 Control \ 를 눌러 강제 종료할 수 없다. 그리고 kill 명령을 이용하더라도 SIGKILL 시그널을 이용하여 강제 종료해야 한다.
- main 쓰레드에서 이 함수를 이용하여 시그널 블록 마스크를 설정하면 그 이후에 생성되는 쓰레드는 시그널 마스크를 상속 받게 되므로 main 에서는 대부분의 시그널을 블록하고 상속받은 각 쓰레드에서 다시 이 함수를 호출하여 세부적으로 설정한다.

쓰레드와 시그널

- 다른 쓰레드로 시그널 전송

```
#include <pthread.h>
#include <signal.h>

int pthread_kill(pthread_t thread, int signo);
```

- 쓰레드간 시그널 전송을 위한 함수로, 첫번째 인자 thread에는 시그널을 전송할 쓰레드 ID를, 두번째 인자 signo 에는 전송할 시그널 번호를 지정한다.
- 사용 예는 다음과 같다.

```
pthread_t tid;

pthread_create( &tid, NULL, thread_func, NULL);
. . .
if( check_flag==1 )
    pthread_kill( tid, SIGUSR1 );
```

쓰레드와 시그널

- 다른 쓰레드로 시그널 전송

```
#include <pthread.h>
#include <signal.h>

int pthread_kill(pthread_t thread, int signo);
```

- 쓰레드간 시그널 전송을 위한 함수로, 첫번째 인자 thread에는 시그널을 전송할 쓰레드 ID를, 두번째 인자 signo 에는 전송할 시그널 번호를 지정한다.

쓰레드와 시그널

- 사용 예는 다음과 같다.

```
pthread_t tid;

pthread_create( &tid, NULL, thread_func, NULL);
. . .
if( check_flag==1 )
    pthread_kill( tid, SIGUSR1 );
```

- 위의 예는 쓰레드 ID가 tid 인 쓰레드로 SIGUSR1 시그널을 전송하는 부분이다. 이 시그널을 수신하는 쓰레드에서는 이 시그널에 대한 핸들러 함수를 먼저 등록해야 한다. 여기서 주의할 점은 tid는 같은 프로세스안에서만 사용할 수 있다는 점이다. 다른 프로세스에서는 tid를 알 수 없다. 따라서 pthread_kill() 함수는 한 프로세스 안에서만 사용 가능하다.

쓰레드와 시그널

■ 시그널 수신

```
#include <pthread.h>  
#include <signal.h>
```

```
int sigwait(const sigset_t *set, int *signo);
```

- 이 함수는 첫번째 인자의 **set**에 설정된 시그널 중 하나가 수신될 때까지 블록상태로 대기한다. 시그널을 수신 받으면 두번째 인자인 **signo** 에 시그널 번호를 반환한다. 함수 명에 **pthread** 라는 키워드가 붙어있지는 않지만 다른 POSIX 쓰레드 함수와 마찬가지로 성공하면 0, 실패 시 에러 코드를 반환한다.

쓰레드와 시그널

- 이 함수는 첫번째 인자의 **set**에 설정된 시그널 중 하나가 수신될 때까지 블록상태로 대기한다. 시그널을 수신 받으면 두번째 인자인 **signo** 에 시그널 번호를 반환한다. 함수 명에 **pthread** 라는 키워드가 붙어있지는 않지만 다른 POSIX 쓰레드 함수와 마찬가지로 성공하면 0, 실패 시 에러 코드를 반환한다.
- 사용 예는 다음과 같다.

```
sigset_t sigset;  
  
sigemptyset(&sigset);  
sigaddset(&sigset, SIGINT);  
sigaddset(&sigset, SIGQUIT);  
sigwait( &sigset, &signo);
```

- 위의 예에서 **sigwait()** 함수를 호출한 쓰레드는 블록 되어 있다가 **SIGINT** 또는 **SIGQUIT** 시그널 중에 하나가 도착하면 **signo** 변수에 어떤 시그널이 도착했는지 반환해준다.



ex09.c 는 각 쓰레드별로 시그널 마스크를 필요에 따라 다르게 설정하는 소스다. 빈 칸을 완성하고 실행해보자.

- 쓰레드 시작 부분에 `pthread_sigmask` 함수를 이용하여 수신할 시그널만 시그널 마스크에서 해제하고 시그널 핸들러를 등록하여 처리한다.

소스코드 (ex09.c)

```

void sig_handler(int signo)
{
    printf("sig_handler : %x received
signal %s\n",
        (int)pthread_self(),
        (signo==SIGUSR1)?"USR1":"USR2");
}

void *thread1_function(void *data)
{
    sigset_t newmask;
    struct sigaction act;
    int i, j;

    act.sa_handler = sig_handler;
    sigfillset(&act.sa_mask);
    act.sa_flags = SA_RESTART;
    sigaction(SIGUSR1, &act, NULL);

    for(i=0; i<10; i++){
        printf("thread1 : tid = %x, i
= %d\n", (int)pthread_self(), i);
        for(j=0; j<999999999; j++);
    }
    return NULL;
}

```

```

void *thread2_function(void *data)
{
    sigset_t newmask;
    struct sigaction act;
    int i, j;

    act.sa_handler = sig_handler;
    sigfillset(&act.sa_mask);
    act.sa_flags = SA_RESTART;
    sigaction(SIGUSR2, &act, NULL);

    for(i=0; i<10; i++){
        printf("thread2 : tid = %x, i
= %d\n", (int)pthread_self(), i);
        for(j=0; j<999999999; j++);
    }
    return NULL;
}

```

```

int main()
{
    int rc, i;
    sigset_t newmask;
    pthread_t thread_t[2];

    if((rc=pthread_create(&thread_t[0],
NULL, thread1_function, NULL))!=0)
    {
        fprintf(stderr, "pthread_create :
error code =%d\n", rc);
        exit(1);
    }

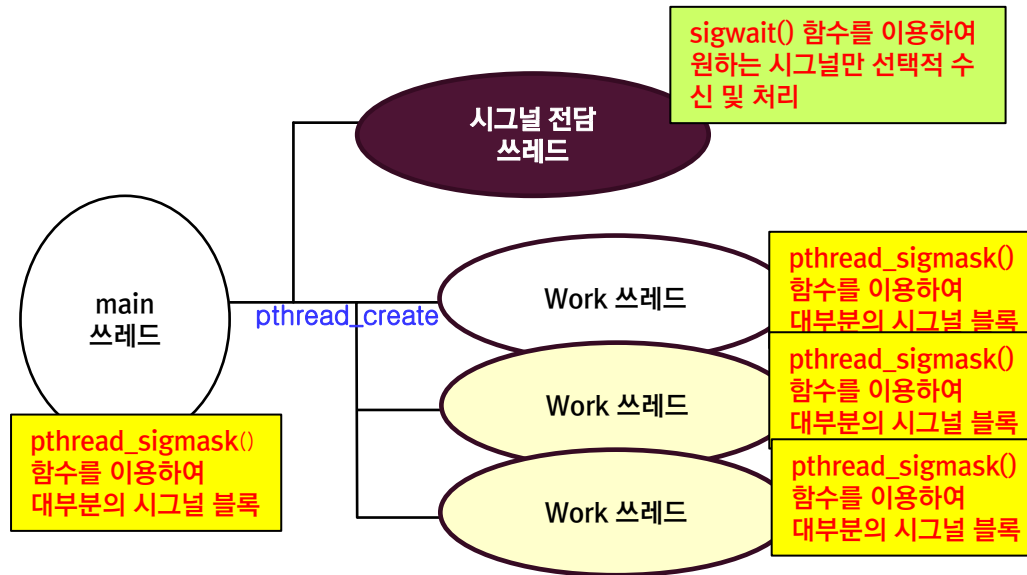
    if((rc=pthread_create(&thread_t[1],
NULL, thread2_function, NULL))!=0)
    {
        fprintf(stderr, "pthread_create :
error code =%d\n", rc);
        exit(1);
    }

    pthread_join(thread_t[0], NULL);
    pthread_join(thread_t[1], NULL);
    return 0;
}

```

시그널 전담 쓰레드

ex10.c는 시그널 처리를 전담하는 쓰레드를 구성하는 방법으로 ex09.c의 방법에 비해 좀더 간단하고 안전하게 시그널 처리를 할 수 있어 현업에서 많이 사용되는 방법이다. 다음 그림은 시그널 전담 쓰레드를 구성하는 경우, 각 쓰레드에서 수행해야 할 내용을 간단하게 표현한 그림이다.



시그널 전담 쓰레드의 구성

소스코드 (ex10.c)

```
void* thread_function(void *x)
{
    int i;

    for (i = 0; i < 10; ++i){
        printf("thread %d (loop #%d)\n", *((int *) x), i);
        sleep(3);
    }
    return (NULL);
}

void* signal_thread(void* arg)
{
    sigset_t sigset;
    int sig;

    sigemptyset(&sigset);
    sigaddset(&sigset, SIGUSR1);
    sigaddset(&sigset, SIGUSR2);
    while (1){
        sigwait(&sigset, &sig);
        switch (sig){
            case SIGUSR1:
                printf("——> caught signal SIGUSR1\n");
                break;
            case SIGUSR2:
                printf("——> caught signal SIGUSR2\n");
                break;
            default:
                printf("unrecognized signal\n");
                break;
        }
    }
}
```

```
int main(void)
{
    pthread_t sigtid, t1, t2;
    int sig, n1 = 1, n2 = 2, rc;
    sigset_t sigset;

    sigfillset(&sigset);
    sigdelset(&sigset, SIGINT);
    pthread_sigmask(SIG_SETMASK, &sigset, NULL);
    if((rc=pthread_create(&sigtid, NULL, signal_thread, NULL))!=0){
        fprintf(stderr, "pthread_create(signal_thread) : rc = %d\n", rc);
        exit(1);
    }
    if((rc=pthread_create(&t1, NULL, thread_function, &n1))!=0){
        fprintf(stderr, "pthread_create(thread) : rc = %d\n", rc);
        exit(1);
    }
    if((rc=pthread_create(&t2, NULL, thread_function, &n2))!=0){
        fprintf(stderr, "pthread_create(thread) : rc = %d\n", rc);
        exit(1);
    }
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(sigtid, NULL);
    return 0;
}
```

실습

- exercise.c는 뮤텁스를 이용하여 쓰레드간의 동기화를 구현한 예제 프로그램이다. thread_function 쓰레드에서 수행 도중 비정상적으로 종료되는 경우, 뮤텁스가 잠긴 채 종료되면 다른 쓰레드에서 문제가 될 수 있으므로 클린업 처리를 하려고 한다. 소스코드를 완성하시오.

소스코드 (exercise.c)

```
#define NUMTHREADS 3
pthread_mutex_t mutexid = PTHREAD_MUTEX_INITIALIZER;
int count=0;

void* thread_function(void* arg)
{
    int i;

    for(i=0; i<5; i++){
        pthread_mutex_lock(&mutexid);
        count++;
        printf("count = %d\n", count);
        sleep(1);
        pthread_mutex_unlock (&mutexid);
    }
}
```

```
int main(void)
{
    pthread_t threads[NUMTHREADS];
    int i;

    for (i=0; i < NUMTHREADS; i++){
        pthread_create(&threads[i],NULL,thread_function,NULL);
    }
    for (i=0; i < NUMTHREADS; i++){
        pthread_join(threads[i], NULL);
    }
    return 0;
}
```